2026 Fall Algorithms — Assignment #1 풀이

---

1. 행렬 곱셈

(a) 기본연산별 횟수

(+=는 덧셈 1회 + 대입 1회로 계산, 반복문 인덱스 연산은 제외)

위치	실행 횟수	곱셈	덧셈	대입
`M[i][j] = A[i][0] * B[0][j]`	n^2	n^2	0	n^2
`M[i][j] += A[i][k] * B[k][j]`	$n^2(n-1)$	$n^2(n-1)$	$n^2(n-1)$	$n^2(n-1)$
합계		n^3	n^3-n^2	n^3

(b) 시간 복잡도

(a)의 기본연산 횟수를 모두 합하면

$$T(n) = n^3 + (n^3 - n^2) + n^3 = 3n^3 - n^2$$

---

2. 점근 표기 증명

(a) $6n^3 - 15n^2 + 20 = O(n^3)$

$n \geq 1$ 일 때 $-15n^2 \leq 0$, $20 \leq 20n^3$ 이므로

$6n^3 - 15n^2 + 20 \leq 6n^3 + 20n^3 = 26n^3$

따라서 $c = 26,\ n_0 = 1$ 로 $T(n) \leq c\,n^3$ 성립. ■

(b) $7n^2 - 4n\log_2 n = \Omega(n^2)$

$n \geq 1$ 일 때 $\log_2 n \leq n$ 이므로 $4n\log_2 n \leq 4n^2$.

$7n^2 - 4n\log_2 n \geq 7n^2 - 4n^2 = 3n^2$

따라서 $c = 3,\ n_0 = 1$ 로 $T(n) \geq c\,n^2$ 성립. ■

(c) $3n^2 + 5n - 2 = \Theta(n^2)$

- **상한**: $n \geq 1$ 일 때 $5n \leq 5n^2$, $-2 \leq 0$ 이므로 $T(n) \leq 8n^2$. ($c_2 = 8, n_0 = 1$)
- **하한**: $n \geq 2$ 일 때 $5n - 2 > 0$ 이므로 $T(n) \geq 3n^2 \geq 2n^2$. ($c_1 = 2, n_0 = 2$)

$n \geq 2$ 에서 $2n^2 \leq T(n) \leq 8n^2$ 이므로 $T(n) = \Theta(n^2)$. ■

---

3. 재귀함수 ($n = 3^k$)

(a) 점화식

반복문이 n번 수행되고 재귀 호출이 3번, 각 호출의 입력 크기는 $n/3$ 이므로

$$T(n) = 3T(n/3) + n, \quad T(1) = 1$$

(b) Iterative substitution

$$\begin{aligned} T(n) &= 3T(n/3) + n \\ &= 3\left[3T(n/9) + n/3\right] + n = 9T(n/9) + 2n \\ &= 9\left[3T(n/27) + n/9\right] + 2n = 27T(n/27) + 3n \\ &\vdots \\ &= 3^i\, T(n/3^i) + i\, n \end{aligned}$$

$i = \log_3 n$ 일 때 $n/3^i = 1$ 이므로

$$T(n) = n\,T(1) + n\log_3 n = n + n\log_3 n$$

$n \geq 3$ 이면 $n \leq n\log_3 n$ 이므로 $T(n) \leq 2n\log_3 n \rightarrow T(n) = O(n\log n)$. ■

(c) Guess & Verification

Guess: $T(n) \leq 2n\log_3 n$ ($n = 3^k,\ k \geq 1$)

Base ($n = 3$): $T(3) = 3T(1) + 3 = 6 \leq 2 \cdot 3 \cdot 1 = 6$ ✓

Inductive step: $T(n/3) \leq 2 \cdot \frac{n}{3}\log_3 \frac{n}{3} = \frac{2n}{3}(\log_3 n - 1)$ 이라 가정하면

$$\begin{aligned} T(n) &= 3T(n/3) + n \\ &\leq 2n(\log_3 n - 1) + n \\ &= 2n\log_3 n - n \\ &\leq 2n\log_3 n \end{aligned}$$

따라서 $T(n) = O(n\log n)$. ■

> 채점 참고: base case를 $n = 1$ 로 잡으면 $\log_3 1 = 0$ 이라 성립하지 않으므로 $n_0 = 3$ 이상으로 잡아야 합니다. (흔한 실수 포인트)

3번 참고: 코드를 문자 그대로 "기본연산" 단위로 해석하는 경우 - 이 경우에도 정답으로 인정

3번은 $T(n) = 3T(n/3) + n$ (재귀 3회 + 선형 작업)을 의도한 문제입니다. 그러나 코드를 다음과 같이 간결하게 쓰면

```python
def fn(n):
    if n <= 1:
        return 1
    else:
        return 3 * fn(n / 3) + n
```

`3 * (...)` 는 곱셈 기본연산 1회, `... + n` 은 덧셈 기본연산 1회로 해석될 여지가 있습니다. 이 경우 `fn(n/3)` 은 **한 번만** 호출되므로 점화식이 달라집니다.

(a) 점화식: 비교 1회, 곱셈 1회, 덧셈 1회 등 재귀 외 작업은 상수 c 이므로

$$T(n) = T(n/3) + c, \quad T(1) = d$$

(b) Iterative substitution

$$T(n) = T(n/3) + c = T(n/9) + 2c = \cdots = T(n/3^i) + i\,c$$

$i = \log_3 n$ 일 때 $T(n) = d + c\log_3 n \rightarrow T(n) = O(\log n)$. ■

(c) Guess & Verification: $T(n) \le C\log_3 n$ ($C = c + d$, $n = 3^k,\ k \ge 1$)

- Base ($n = 3$): $T(3) = c + d \le C \cdot 1$ ✓
- Step: $T(n) = T(n/3) + c \le C(\log_3 n - 1) + c = C\log_3 n - d \le C\log_3 n$ ✓

따라서 $T(n) = O(\log n)$. ■

정리

해석	점화식	결과
의도한 해석 (재귀 3회 + 선형 작업)	$T(n) = 3T(n/3) + n$	$\Theta(n \log n)$
기본연산 해석 (`3 * fn(n/3) + n`)	$T(n) = T(n/3) + c$	$\Theta(\log n)$

> **출제 관련 안내**: 위와 같이 `3 * fn(n/3) + n` 형태의 코드는 곱셈·덧셈이 기본연산 1회로 해석되어 의도와 다른 점화식이 나올 수 있습니다. 따라서 본 문제지는 반복문과 재귀 호출 3회를 코드에 명시하여 $T(n) = 3T(n/3) + n$ 으로 해석되도록 수정하였으며, 향후 시험에서도 이와 같은 오해의 소지가 없도록 코드의 연산 구조를 명확히 하거나 점화식 형태를 문제에 직접 제시하여 출제할 예정입니다.

---

4. Master Theorem

$T(n) = aT(n/b) + f(n)$ 에서 $n^{\log_b a}$ 와 $f(n)$ 을 비교.

	a, b	$n^{\log_b a}$	$f(n)$	Case	결과
(a)	2, 3	$n^{0.63}$	n	3	$\Theta(n)$
(b)	1, 3	$n^0 = 1$	n	3	$\Theta(n)$
(c)	4, 2	n^2	n	1	$\Theta(n^2)$

	a, b	$n^{\log_b a}$	$f(n)$	**Case**	**결과**
(d)	1, 2	$n^0 = 1$	1	2	$\Theta(\log n)$
(e)	4, 2	n^2	n^2	2	$\Theta(n^2 \log n)$
(f)	8, 2	n^3	n^3	2	$\Theta(n^3 \log n)$
(g)	5, 5	n	n^2	3	$\Theta(n^2)$

세부 설명

- **(a)** $\log_3 2 \approx 0.63$. $f(n) = n = \Omega(n^{0.63+\varepsilon})$ ($\varepsilon = 0.3$). Regularity: $2 \cdot \frac{n}{3} \leq \frac{2}{3} f(n)$, $c = 2/3 < 1$ ✓ → Case 3.
- **(b)** $\log_3 1 = 0$. $f(n) = n = \Omega(n^{0+\varepsilon})$ ($\varepsilon = 1$). Regularity: $\frac{n}{3} \leq \frac{1}{3} f(n)$ ✓ → Case 3.
- **(c)** $\log_2 4 = 2$. $f(n) = n = O(n^{2-\varepsilon})$ ($\varepsilon = 1$) → Case 1.
- **(d)** $\log_2 1 = 0$. $f(n) = 1 = \Theta(n^0)$ → Case 2, $\Theta(n^0 \log n)$.
- **(e)** $\log_2 4 = 2$. $f(n) = \Theta(n^2)$ → Case 2.
- **(f)** $\log_2 8 = 3$. $f(n) = \Theta(n^3)$ → Case 2.
- **(g)** $\log_5 5 = 1$. $f(n) = n^2 = \Omega(n^{1+\varepsilon})$ ($\varepsilon = 1$). Regularity: $5 \cdot \left(\frac{n}{5}\right)^2 = \frac{1}{5} n^2$, $c = 1/5 < 1$ ✓ → Case 3.